Rat in a maze problem :-

https://www.geeksforgeeks.org/problems/rat-in-a-maze-problem/1

```
class Solution {
  public:

    // Directions: Down, Left, Right, Up
    string dir = "DLRU";
    int dr[4] = {1, 0, 0, -1};
    int dc[4] = {0, -1, 1, 0};

    // Check if a cell is valid (inside the maze and open)
    bool isValid(int r, int c, int n, vector<vector<int>>& maze) {
        return r >= 0 && c >= 0 && r < n && c < n && maze[r][c];
    }

    // Function to find all valid paths
    void findPath(int r, int c, vector<vector<int>>& maze, string& path,
                  vector<string>& res) {
        int n = maze.size();

        // If destination is reached, store the path
        if (r == n - 1 && c == n - 1) {
            res.push_back(path);
            return;
        }

        // Mark current cell as blocked
        maze[r][c] = 0;

        for (int i = 0; i < 4; i++) {
            int nr = r + dr[i], nc = c + dc[i];
            if (isValid(nr, nc, n, maze)) {
                path.push_back(dir[i]);

                // Recurse to next cell
                findPath(nr, nc, maze, path, res);

                // Backtrack
                path.pop_back();
            }
        }

        // Unmark current cell
```

```
        maze[r][c] = 1;
    }

    // Function to find all paths and return them
    vector<string> ratInMaze(vector<vector<int>>& maze) {
        vector<string> result;
        int n = maze.size();
        string path = "";

        if (maze[0][0] != 0 && maze[n - 1][n - 1] != 0) {

            // Start DFS from (0,0)
            findPath(0, 0, maze, path, result);
        }

        return result;
    }
};
```

https://www.geeksforgeeks.org/dsa/n-queen-problem-backtracking-3/

```
// C++ Program to solve the n-queens problem
#include <bits/stdc++.h>
using namespace std;

// Function to check if it is safe to place
// the queen at board[row][col]
int isSafe(vector<vector<int>>& mat,
                    int row, int col) {
    int n = mat.size();
    int i, j;

    // Check this col on upper side
    for (i = 0; i < row; i++)
        if (mat[i][col])
            return 0;

    // Check upper diagonal on left side
    for (i = row-1, j = col-1; i >= 0 &&
         j >= 0; i--, j--)
        if (mat[i][j])
            return 0;
```

```
    // Check upper diagonal on right side
    for (i = row-1, j = col+1; j < n &&
         i >= 0; i--, j++)
        if (mat[i][j])
            return 0;

    return 1;
}

int placeQueens(int row, vector<vector<int>>& mat) {
    int n = mat.size();

    // base case: If all queens are placed
    // then return true
    if(row == n) return 1;

    // Consider the row and try placing
    // queen in all columns one by one
    for(int i = 0; i < n; i++){

        // Check if the queen can be placed
        if(isSafe(mat, row, i)){
            mat[row][i] = 1;
            if(placeQueens(row + 1, mat))
                return 1;
            mat[row][i] = 0;
        }
    }
    return 0;
}

// Function to find the solution
// to the N-Queens problem
vector<int> nQueen(int n) {

    // Initialize the board
    vector<vector<int>> mat(n, vector<int>(n, 0));

    // If the solution exists
    if(placeQueens(0, mat)){

        // to store the columns of the queens
        vector<int> ans;
        for(int i = 0; i < n; i++){
            for(int j = 0; j < n; j++){
```

```
                if(mat[i][j]){
                    ans.push_back(j + 1);
                }
            }
        }
        return ans;
    }
    else return {-1};
}

int main() {
    int n = 4;
    vector<int> ans = nQueen(n);
    for(auto i: ans){
        cout << i << " ";
    }
    return 0;
}
```